Reconstructing OpenAI's nearly quartic query separation

Comparing candidates and predicting a hidden address

Will Cook · November 2026

Abstract. OpenAI proved that no exponent below four bounds randomized query complexity in terms of quantum query complexity for all total Boolean functions. We reconstruct the separating family. A quantum algorithm compares candidate certificates before verifying them. It samples each pivot in proportion to its probability of beating earlier pivots, without computing those weights. With this choice even arbitrary comparison biases between losers cancel. Classically, a query to one bit of the winning certificate must first name its address. A capped simulation turns this choice into a prediction of many independent hard answers.

1. The fourth power

For a total Boolean function f, let $R(f)$ and $Q(f)$ denote its bounded-error randomized and quantum bit-query complexities. Aaronson, Ben-David, Kothari, Rao and Tal proved that $O(Q(f)^4)$ queries suffice even for a deterministic algorithm and conjectured $R(f) = O(Q(f)^3)$ [1, Conjecture 2]. Earlier cheat-sheet constructions attain powers approaching three [2–4]. OpenAI [5] showed that the quartic bound is sharp in its exponent.

Start with m strings of m bits, arranged in columns as in earlier pointer functions [6; 7]. To win, a column must be all ones and exhibit a zero in every rival. It keeps the row pointers to these zeros together in a certificate cell. Each column has a whole array of such cells. The answers to several instances of a partial function P, written in order, give the address of the cell to inspect. This is the cheat-sheet idea of Aaronson, Ben-David and Kothari [2].

Take P to be k-fold Forrelation on k blocks of m bits. At fixed k a quantum algorithm can decode a promised address in polylogarithmically many queries. To compare two columns it follows just the pointers they direct at each other. When the winner's address is decoded correctly, its pointer reaches a zero. Even if the rival decodes the wrong address, its return pointer reads a one. Such comparisons give a short list of candidates in $\widetilde{O}_k(\sqrt{m})$ queries. To check a candidate, search for a bad pointer, a zero in its own column, or a fault in the circuit transcripts certifying its address. The circuit checks the promise and value of P with $\widetilde{O}_k(m)$ gates, so verification costs another $\widetilde{O}_k(\sqrt{m})$ queries.

On the classical side even one promised instance of P can require $\widetilde{\Omega}_k(m^{1-1/k})$ queries [3]. A query to the winning cell predicts all the answers in its address. Hiding that cell in a random column supplies a further factor of m. Up to logarithms, the quantum upper and randomized lower bounds are $(m, m^{3-1/k})$ for the balanced AND–OR cheat sheets using the same P [2, § 3], and $(\sqrt{m}, m^{2-1/k})$ here. The classical bound loses a factor of m, yet the separation exponent rises from $3 - 1/k$ to $4 - 2/k$. Already $k = 3$ excludes the cube.

Theorem 1 (OpenAI [5, Theorem 1.1], with explicit bounds from this reconstruction). *For each integer $k \geq 2$ there are total Boolean functions $F_{k,m}$ indexed by powers of four $m \geq 4$ such that, for a constant C_k and $\ell = \log_2 m$,*

$$Q(F_{k,m}) \leq C_k \sqrt{m}\, \ell^6, \qquad R(F_{k,m}) \geq \frac{m^{2-1/k}}{400\ell^2}. \tag{1}$$

The quantum bound holds for all such m and the randomized bound for all sufficiently large m at fixed k. The optimal universal exponent in $R(f) = O((1 + Q(f))^\alpha)$ for total Boolean f is four.[1]

[1] We use worst-case bit-query caps and error at most 1/3. For partial functions the cap applies on every input and branch, the error bound on the promise.

2. The certificate

In addition to its string $X_i \in \{0,1\}^m$, column $i \in [m]$ holds $r = 20 \log_2 m$ inputs $A_{i,1}, \ldots, A_{i,r}$ to P, and one certificate cell $B_{i,s}$ for each $s \in \{0,1\}^r$. Even with one cell for every possible address there are just $2^r = m^{20}$ cells per column.

A certificate has to do more than point to zeros. The address itself could be false, or one of the partial-function answers could be undefined. Fix an exact fan-in-two Boolean circuit $\mathcal{C}$ with two outputs, a domain bit and the value of P on its domain. Each cell holds a proposed evaluation of $\mathcal{C}$ for each address input, one bit per non-input gate. It also holds one ℓ-bit row pointer $p_{i,s,j}$ into each column $j \neq i$. Every pointer word names a row.

Column i **wins** if $X_i = 1^m$, all its address inputs belong to the domain of P, and the cell with label

$$s_i = (P(A_{i,1}), \ldots, P(A_{i,r}))$$

contains the true transcripts and points to a zero in every other column. The function outputs one if a column wins, zero otherwise. All input bits are freely chosen. An off-promise column loses, whatever is written in its cells. Two winners are impossible because neither all-one column could point to a zero in the other.

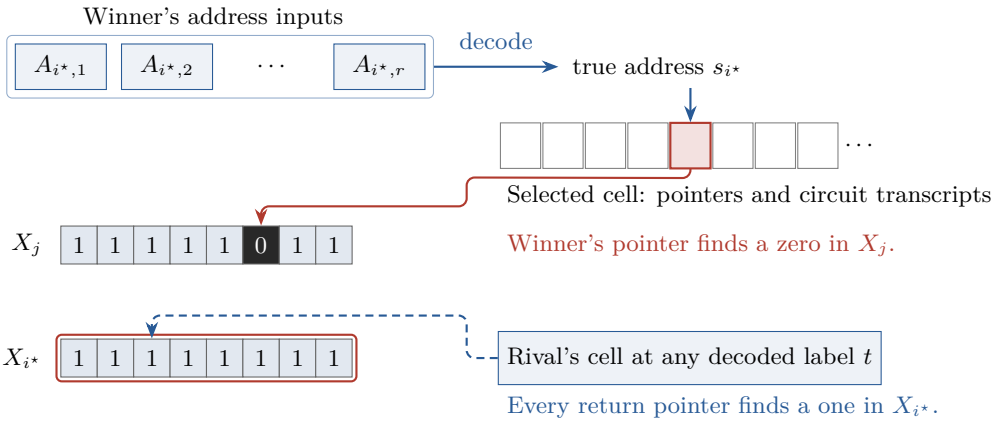


Figure 1: A comparison with the winning column $i^\star$. Its selected cell points to a zero in X_j. The return pointer reads a one in $X_{i^\star}$, whichever cell the rival decodes.

If a claimed address s is false, where can a verifier catch it? Test the m bits of X_i and follow the $m-1$ outward pointers. For each address input test the transcript against every gate of $\mathcal{C}$ and test the two outputs against $(1, s_h)$. A consistent transcript must be the true evaluation, by induction in circuit order. A fabricated transcript therefore fails at some gate, and a true transcript for an undefined input or wrong answer fails an output test. In particular a losing column has a defect at *every* address. This gives the verifier a search problem with

$$K = m + (m-1) + r(S+2) \tag{2}$$

tests, where S counts non-input gates. Reading a gate and its two inputs costs at most three queries, and following a pointer costs $\ell + 1$. We fix this circuit for both query bounds, since its gate list determines the cell format.

We use the following quantitative bounds for k-fold Forrelation [5, Lemma 2.1], with the promise specified in §5. A reversible decoder for P with error η on its domain costs $O_k(1 + \log(1/\eta))$ queries on every input. An exact circuit as above has $S = O_k(m\ell^2)$, so $K = O_k(m\ell^3)$ and the entire input is polynomial in m at fixed k. For the randomized argument let L be a query budget insufficient to solve one instance of P. For fixed k and sufficiently large m we can take

$$L := \lfloor m^{1-1/k}/\ell^2 \rfloor < R(P). \tag{3}$$

3. Finding a winner by comparison

3.1 What one comparison reveals

Decode the two column addresses, follow the two pointers directed at each other, and favour the column whose pointer finds a zero when the other finds a one. Break ties by smaller index. Suppose $i^\star$ wins. With its address decoded correctly, its pointer finds a zero. Whichever cell its rival decodes, the return pointer goes into $X_{i^\star} = 1^m$ and reads a one. This comparison works even when none of the rival's address inputs obeys the promise.

Write ε for the allowed error in decoding a promised column. Running the decoder on each address bit with error ε/r suffices. Use the same coherent circuit for each unordered pair, retaining both decoded addresses, and assign a loss to self-comparison. Since the two directions now denote complementary outcomes of *one* comparison, the probabilities $a_{ij} = \Pr[i \text{ defeats } j]$ on a fixed input satisfy

$$a_{ij} + a_{ji} = 1 \quad (i \neq j), \qquad a_{ii} = 0. \tag{4}$$

Only the winning column has a dominance guarantee, $a_{i^\star j} \geq 1 - \varepsilon$ for every $j \neq i^\star$. The probabilities between losers may lie anywhere in $[0, 1]$. One comparison or its inverse costs $c = O_k(r \log(r/\varepsilon) + \ell)$ queries.

3.2 A tournament with probabilistic comparisons

The tournament returns its successive pivots as the candidate list [5, Proposition 3.2]. First suppose the comparison probabilities are known. We seek a rule for drawing a pivot under which losing candidates drop out rapidly and a dominant candidate survives until drawn.

For fixed comparison outcomes one can keep precisely the candidates that beat all previous pivots and draw uniformly from this set [8, § 6.1]. For fractional comparisons the analogue of membership is the probability of beating those pivots in fresh comparisons. After pivots $\rho_1, \dots, \rho_n$, this gives weights

$$w_i = \prod_{h=1}^{n} a_{i\rho_h}, \qquad W = \sum_i w_i.$$

Initially all weights are one. A pivot j updates w_i to $w_i a_{ij}$ and removes itself. How should we draw j once the weights are unequal? For a pivot law q, the pair i, j contributes $w_i q_j a_{ij} + w_j q_i a_{ji}$ to the expected remaining mass. A uniform pivot leaves the larger weight on one direction. To cancel the bias we need equal coefficients, $w_i q_j = w_j q_i$. Drawing j with probability w_j/W achieves this for every pair. Hence, while $W > 0$,

$$\mathbb{E}[W_{\text{new}} \mid w] = \frac{1}{W} \sum_{i,j} w_i w_j a_{ij} = \frac{W^2 - \sum_i w_i^2}{2W} \leq \frac{W}{2}. \tag{5}$$

The sum pairs two independent draws from the weights. On distinct draws either is equally likely to defeat the other, and coincident draws contribute zero.

Stop on extinction, keeping $W = 0$ thereafter. After T steps the expected mass is at most $m2^{-T}$. An unselected dominant candidate, however, still has weight at least $(1 - \varepsilon)^T \geq 1 - T\varepsilon$. Take $T = \lceil \log_2(1000m) \rceil$ to make the first quantity small and $\varepsilon = 1/(1000T)$ to keep the second above $1/2$. Missing this candidate now requires more than half a unit of weight to survive. By Markov's inequality that happens with probability at most $2m2^{-T}$.

The quantum algorithm has only the reversible comparison circuit and the list of pivots drawn so far. To draw its next pivot, it prepares a uniform superposition of i and compares i with every listed pivot in fresh workspaces, coherently controlled by the candidate indices. If $|\phi(i, \rho_b)\rangle$ is the entire state of one comparison, the preparation produces

$$\frac{1}{\sqrt{m}} \sum_i |i\rangle \bigotimes_{b=1}^{n} |\phi(i, \rho_b)\rangle.$$

Flag the event that *all* comparisons favour i. Within a fixed label the independent comparison registers multiply the success probabilities. Across distinct labels their masses add. Consequently

$$\Pr[i \text{ and flag}] = \frac{w_i}{m}, \qquad \Pr[\text{flag}] = \frac{W}{m}, \qquad \Pr[i \mid \text{flag}] = \frac{w_i}{W}. \tag{6}$$

The same half unit of surviving weight pays for this draw. Until $i^\star$ is listed, its contribution gives flag probability $W/m > 1/(2m)$. Amplify the flag with this threshold and failure tolerance ε. On success append the measured label to the list. Stop on failure or after T draws. The finite schedule costs $O(\sqrt{m}\log(1/\varepsilon))$ calls to the preparation and its inverse on every branch. Grover rotations preserve the direction of the flagged component, so every successful trial has *exactly* the label distribution in (6) [9; 10; 5, Lemma 3.1].

Thus at any history before the winner is listed, the algorithm stops with probability at most ε and otherwise has precisely the ideal next-pivot law. Coupling at each such history gives

$$\Pr[i^\star \text{ absent from the list}] \le 2m2^{-T} + T\varepsilon. \tag{7}$$

Rebuilding at most T comparisons per preparation for at most T stages costs $O(c\sqrt{m}\,T^2 \log T)$ queries on every input.

3.3 Paying for verification

Once the list is drawn, decode a fresh address for each listed column and search its K local tests for a defect. Accept if at least one of these searches finds none. A uniform test flags a defect with probability at least $1/K$ whenever any test fails. With threshold $1/K$ the bounded search costs $O(\ell\sqrt{K}\log(1/\varepsilon))$ queries per column and misses an existing defect with probability at most ε.

On a one-input, listing the winner and decoding its address correctly suffice for acceptance, since every test there passes. Rejection therefore has probability at most (7) plus ε. On a zero-input the tournament may return any list at all: comparison never reads a transcript, and a column with false transcripts might beat every rival. But every listed column fails some test at *every* address (§2). Conditional on the list, the fresh searches give false-acceptance probability at most $T\varepsilon$. Both errors are below $1/3$.

The cost of checking the list is $O(T(c + \ell\sqrt{K}\log T))$, including fresh address decoding. With $T = O(\ell)$, $c = O_k(\ell \log \ell)$ and $K = O_k(m\ell^3)$, this and the tournament cost sum to at most $C_k\sqrt{m}\,\ell^6$. The number of candidates sets the first square-root cost and the size of a single certificate sets the second.

4. A cell query predicts an address

4.1 Planting one certificate

For the **dummy input** $Y^{(0)}$, choose the row J_i of the sole zero in X_i independently and uniformly in each column. Draw all address inputs independently from a law μ on the promise of P, independently also of the zero positions, and set every cell bit to zero. Every column loses. Choose I uniformly and independently of the entire dummy input. For the **planted input** $Y^{(1)}$, erase the zero in column I and fill its true cell B_{I,s_I} with the correct transcripts and the pointers J_j to the other zeros. Column I wins.

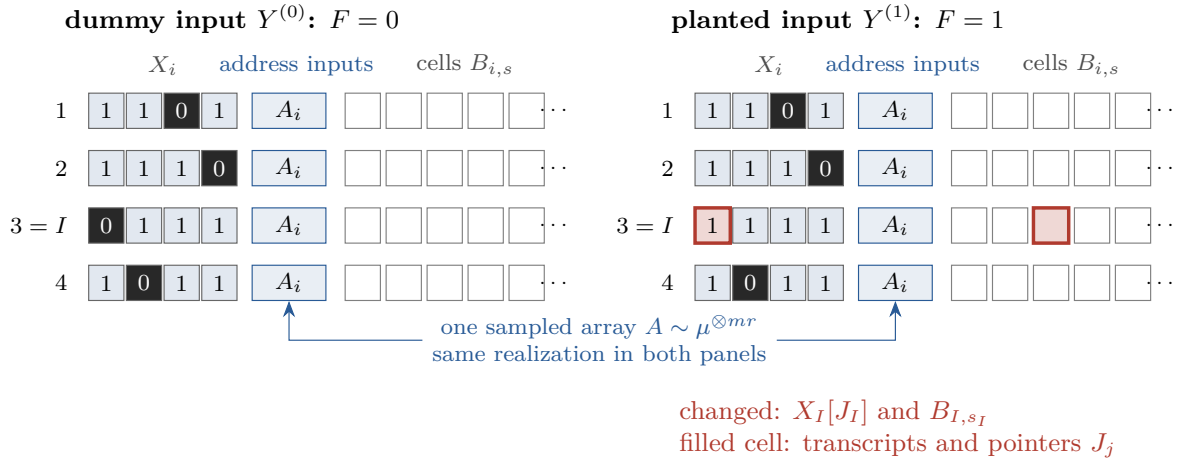


Figure 2: One draw of the coupled inputs, with $I = 3$. All of the address problems have the same inputs in the two panels. The planted winner differs from the corresponding losing column only in its bit strip and its addressed cell.

Run a deterministic tree on the dummy board. It sees the same replies on the planted board until it queries either the erased zero or the filled cell. The acceptance gap is at most the sum of the two discovery probabilities on the dummy run, which knows nothing about I [2, Lemma 6; 11, § 5.1]. To reach the cell, the tree can investigate a few columns deeply and hope I is among them, or it can try addresses in many columns after learning little about each.

For any integer $1 \le L \le m$ with $L < R(P)$, minimax gives a μ under which every deterministic depth-L tree computes P with success at most $\beta = 2/3$ [12]. Give the tree on the board $G = \lfloor mL/100 \rfloor$ queries. We first bound the chance of bypassing the address search altogether.

4.2 Searching for the erased zero

If the tree discovers Z zeros, the probability that one of them belongs to column I is exactly $\mathbb{E}Z/m$. Before discovery, a column's zero is uniform among the rows not yet queried there, even given the entire transcript. Until half the rows have been queried, a query makes a new discovery with probability at most $2/m$. We expect at most $2G/m$ such early discoveries. Each remaining discovery costs at least $m/2$ queries to its column, so the run has at most $2G/m$ late discoveries. Hence

$$\Pr[\text{query } X_I[J_I]] \le \frac{4G}{m^2}. \tag{8}$$

4.3 Predicting the filled cell

To query a bit of $B_{i,s}$ the tree has to announce s in the query index. At the right cell this is a prediction of all r answers in column i. Call a column *heavy* if the full dummy run makes more than L queries to its address inputs, counting repetitions. There are at most G/L such columns on any run. Because I is independent of that run,

$$\Pr[I \text{ heavy}] \le \frac{G}{mL}.$$

In a light column, a query to the true cell names r hard answers after at most L queries to their inputs. It is tempting to condition on the column being light and apply a direct-product bound. But the answers seen by the tree may determine whether it continues in that column. Conditioning on its final query count can change the input law.

Fix a column i and query time $\tau \le G$. Embed the predictor's unknown tuple as column i's address inputs on a dummy board, and sample all other data independently of the tuple. Simulate the tree through time τ, paying only for queries to these address inputs. Stop before the $(L+1)$-st such query in total. If query τ is in a cell $B_{i,s}$, output s before reading the cell. If the query at that time has any other form, or if the simulation aborts or halts earlier, output a fixed vector.

The total cap gives the predictor at most L queries to *each* target input, on every branch. Under $\mu^{\otimes r}$ its success probability is therefore at most β^r by the separate-cap direct-product bound. This bound allows adaptive interleaving and side information and coins jointly independent of the tuple [13, § 3; 5, Lemma 4.1].[2]

On a run in which i ends up light, every prefix was within budget. If query τ on that run hits B_{i,s_i}, the predictor reaches it and outputs s_i. This event inclusion gives the joint bound

$$\Pr[i \text{ light and query } \tau \text{ is in } B_{i,s_i}] \;\le\; \Pr[\text{predictor outputs } s_i] \;\le\; \beta^r. \tag{9}$$

For each fixed column there are at most G opportunities to make this prediction. Average over the independent choice of I and add the chance of a heavy column to obtain

$$\Pr[\text{query } B_{I,s_I}] \le \frac{G}{mL} + \frac{1}{m}\sum_{i=1}^{m} G\beta^r = \frac{G}{mL} + G\beta^r. \tag{10}$$

4.4 The randomized bound

Combining (8) and (10), the gap between the acceptance probabilities on the two laws is at most

$$\frac{4G}{m^2} + \frac{G}{mL} + G(2/3)^r < \frac{1}{3}. \tag{11}$$

Here $G \le mL/100 \le m^2/100$, and $r = 20\log_2 m$ makes $(2/3)^r \le m^{-10}$ small enough for all G guesses. Averaging over coins gives (11) for a randomized tree, while correctness with error at most $1/3$ on both supports would force a gap of at least $1/3$. Thus every such construction requires more than $\lfloor mL/100 \rfloor$ randomized queries. For (3) at fixed k, the erased-zero term is $O(L/m) = o(1)$ and the lucky-address term also vanishes. The term that fixes the budget is $G/(mL)$, the bound on the fraction of deeply queried columns.

[2] At a fixed transcript the posterior factors across inputs. Give each input its best probability of a correct answer with its remaining queries. A query to one input leaves the other factors fixed and cannot increase the chosen factor in expectation, since the query was already an available first move. The product gives the separate-cap bound.

5. Forrelation and the optimal exponent

The partial function supplying these parameters is the Walsh–Hadamard form of k-fold Forrelation [3; 14]. Let H be the normalized Walsh–Hadamard matrix of order m, $u = m^{-1/2}(1, \dots, 1)^{\mathsf{T}}$, and D_z the diagonal matrix of a sign vector. For k such vectors of length m, put

$$\Phi = u^{\mathsf{T}} D_{z_1} H D_{z_2} H \cdots H D_{z_k} u, \qquad \delta = 2^{-5k}.$$

Set $P = 1$ when $\Phi \geq \delta$, $P = 0$ when $|\Phi| \leq \delta/2$, and leave P undefined elsewhere. Applying the unitary product to u and projecting onto u succeeds with probability Φ^2. For fixed k the two promised cases have a constant gap, giving the claimed decoder by repetition with trial registers retained. This experiment loses the sign: a large negative Φ can pass the projection test and still be outside the promise. For the exact circuit, compute the signed numerator by $k - 1$ unnormalized fast Walsh–Hadamard transforms on integers of $O_k(\ell)$ bits and compare it to the thresholds above. This yields both domain and value bits in $O_k(m\ell^2)$ fan-in-two gates [5, Lemma 2.1]. The lower bound of Bansal and Sinha [3, Corollary 1.4] gives (3). Fixing this circuit in the cells defines $F_{k,m}$.

For sufficiently large m, the floor in (3) gives $L \geq m^{1-1/k}/(2\ell^2)$ and $\lfloor mL/100 \rfloor \geq mL/200$, proving the randomized inequality of (1). Now fix $0 \leq \alpha < 4$. Choose $k \geq 2$ with $k > 2/(4-\alpha)$ and hold it fixed as m grows through powers of four. By (1),

$$\frac{R(F_{k,m})}{(1 + Q(F_{k,m}))^{\alpha}} \;\geq\; \frac{1}{400(1 + C_k)^{\alpha}} \, \frac{m^{2-1/k-\alpha/2}}{(\log_2 m)^{2+6\alpha}} \;\longrightarrow\; \infty.$$

For every power below four, then, one fixed choice of k gives a family violating that power. Together with the quartic upper bound, this gives optimal exponent four.

References

[1] S. Aaronson, S. Ben-David, R. Kothari, S. Rao and A. Tal. *Degree vs. approximate degree and quantum implications of Huang's sensitivity theorem*. Proceedings of the 53rd annual ACM symposium on theory of computing (STOC 2021), 1330–1342, 2021, arXiv:2010.12629.

[2] S. Aaronson, S. Ben-David and R. Kothari. *Separations in query complexity using cheat sheets*. Proceedings of the 48th annual ACM symposium on theory of computing (STOC 2016), 863–876, 2016, arXiv:1511.01937v2.

[3] N. Bansal and M. Sinha. *k-Forrelation optimally separates quantum and classical query complexity*. Proceedings of the 53rd annual ACM symposium on theory of computing (STOC 2021), 1303–1316, 2021, arXiv:2008.07003v3.

[4] A. A. Sherstov, A. A. Storozhenko and P. Wu. *An optimal separation of randomized and quantum query complexity*. SIAM Journal on Computing, 52 (2), 525–567, 2023, arXiv:2008.10223v4, 29 January 2023.

[5] OpenAI. *A nearly quartic separation between randomized and quantum query complexity*. 2026, Preprint dated 5 October 2026. Online source.

[6] M. Göös, T. Pitassi and T. Watson. *Deterministic communication vs. partition number*. SIAM Journal on Computing, 47 (6), 2435–2450, 2018, preliminary version in FOCS 2015.

[7] A. Ambainis, K. Balodis, A. Belovs, T. Lee, M. Santha and J. Smotrovs. *Separations in query complexity based on pointer functions*. Journal of the ACM, 64 (5), 32:1–32:24, 2017, arXiv:1506.04719v3, 26 October 2015.

[8] A. Ambainis, J. Iraids and M. Kokainis. *Near-optimal separations of certificate complexity from randomized and quantum query complexity*. 2026, arXiv:2609.11664v3, Section 6.1. https://arxiv.org/abs/2609.11664v3.

[9] M. Boyer, G. Brassard, P. Høyer and A. Tapp. *Tight bounds on quantum searching*. Fortschritte der Physik, 46 (4–5), 493–505, 1998, arXiv:quant-ph/9605034v1, Section 4.

[10] G. Brassard, P. Høyer, M. Mosca and A. Tapp. *Quantum amplitude amplification and estimation*. Quantum computation and quantum information, Contemporary mathematics, 305, 53–74, American Mathematical Society, 2002, arXiv:quant-ph/0005055v1.

[11] S. Ben-David. *A super-Grover separation between randomized and quantum query complexities*. 2015, arXiv:1506.08106.

[12] A. C.-C. Yao. *Probabilistic computations: Toward a unified measure of complexity*. Proceedings of the 18th annual symposium on foundations of computer science (FOCS 1977), 222–227, 1977.

[13] A. Drucker. *Improved direct product theorems for randomized query complexity*. Computational Complexity, 21 (2), 197–244, 2012, arXiv:1005.0644v2.

[14] S. Aaronson and A. Ambainis. *Forrelation: A problem that optimally separates quantum from classical computing*. SIAM Journal on Computing, 47 (3), 982–1038, 2018, arXiv:1411.5729v1, Section 3.2.

Reconstruction. The complete argument includes the uniform estimates for a growing parameter. The theorem index links the formal statements and Lean proofs. Will Cook directed the reconstruction. AI systems drafted and revised the manuscripts and Lean development.